



Reproducibility Companion Paper: Blind Natural Video Quality Prediction via Statistical Temporal Features and Deep Spatial Features

Jari Korhonen
School of Computer Science and
Software Engineering
Shenzhen University
Shenzhen, Guangdong, China
jari@szu.edu.cn

Yicheng Su
School of Computer Science and
Software Engineering
Shenzhen University
Shenzhen, Guangdong, China
suyicheng1995@gmail.com

Junyong You
Norwegian Research Centre (NORCE)
Bergen, Norway
juyo@norce-research.no

Steven Hicks
Simula Metropolitan Center for
Digital Engineering (SimulaMet)
Oslo, Norway
steven@simula.no

Cise Midoglu
Simula Metropolitan Center for
Digital Engineering (SimulaMet)
Oslo, Norway
cise@simula.no

ABSTRACT

Blind natural video quality assessment (BVQA), also known as no-reference video quality assessment, is a highly active research topic. In our recent contribution titled “Blind Natural Video Quality Prediction via Statistical Temporal Features and Deep Spatial Features” published in ACM Multimedia 2020, we proposed a two-level video quality model employing statistical temporal features and spatial features extracted by a deep convolutional neural network (CNN) for this purpose. At the time of publishing, the proposed model (CNN-TLVQM) achieved state-of-the-art results in BVQA. In this paper, we describe the process of reproducing the published results by using CNN-TLVQM on two publicly available natural video quality datasets.

CCS CONCEPTS

• **Computing methodologies** → *Computer vision tasks*.

KEYWORDS

Video quality assessment; Human visual system; Machine learning; Convolutional neural network

ACM Reference Format:

Jari Korhonen, Yicheng Su, Junyong You, Steven Hicks, and Cise Midoglu. 2021. Reproducibility Companion Paper: Blind Natural Video Quality Prediction via Statistical Temporal Features and Deep Spatial Features. In *Proceedings of the 29th ACM Int'l Conference on Multimedia (MM '21)*, Oct. 20–24, 2021, Virtual Event, China. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3474085.3477933>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MM'21, October 20–24, 2021, Virtual Event, China.

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-8651-7/21/10...\$15.00
<https://doi.org/10.1145/3474085.3477933>

1 INTRODUCTION

The perceived quality of user-generated video content is typically affected by several different types of natural distortions, such as motion blur, sensor noise, flickering, and camera shakiness. Since the relative impact of different distortions on the perceived overall quality is a highly dynamic and subjective by nature, blind natural video quality assessment (BVQA) is a challenging research problem. In image quality assessment (IQA), impressive results have been achieved by using deep convolutional neural network (CNN) [1, 8, 22]. Several deep VQA models have been proposed in the prior art as well [4, 12, 19, 23, 24]. However, 2-dimensional CNN models are not able to capture typical temporal distortions in user-generated videos, such as flickering and shakiness. This was our motivation to combine hand-crafted statistical motion features, first introduced in a two-level video quality model (TLVQM) [9], with spatial features extracted by CNN. The resulting model, CNN-TLVQM, was published in ACM Multimedia 2020 [11].

BVQA is currently a very active research topic, and several new BVQA methods have been proposed recently. Unfortunately, reproducibility problems concerning BVQA methods have also been reported (see e.g. [5]). Apparently, there is a need to improve transparency and reproducibility in BVQA research, and this has motivated us to prepare a simple-to-use reproducibility package for CNN-TLVQM, available under [10]. In [11], we compared the performance of CNN-TLVQM against relevant baseline methods on two commonly used natural video quality datasets, KoNViD-1k [7] and LIVE-VQC [20]. In this paper, we summarize the study on a high level. We also give the necessary instructions for reproducing the results for CNN-TLVQM, including the source code for our implementation and references for downloading the third-party datasets used in the experiments. We do not include the benchmark methods developed by other researchers in this study since the performance of those methods has already been verified independently in other studies, and it would be very time-consuming to extract the features for some of the benchmark methods.

Replication of the results in [11] requires the following four steps:

- (1) Setup and installation, including the preparation of datasets.
- (2) Training the CNN-based feature extractor.
- (3) Extracting CNN-TLVQM features from the video sequences used for training and testing.
- (4) Training and testing the video quality model using the extracted features.

A high-level illustration of the steps is shown in Figure 1. We elaborate on Step 1 in Section 2, and Steps 2-4 in Section 3. In Sections 4 and 5, we provide notes on the reviewing process, runtime benchmarks, and concluding remarks.

2 SETUP AND INSTALLATION

The implementation of CNN-TLVQM is written entirely in Matlab. The model was developed using Matlab version R2018b running on the Windows 10 operating system. Other Matlab versions and operating systems may also be used, assuming they support the required features listed in Section 2.1. The Matlab scripts used in CNN-TLVQM should be located in one directory, and all of the output files will be generated in the same directory.

2.1 (Step 1a) Environment Installation

The following Matlab toolboxes and add-ons are required for running the experiments:

- Image Processing Toolbox [16]
- Statistics and Machine Learning Toolbox [18]
- Deep Learning Toolbox [14]
- Pre-trained ResNet-50 model [15]
- Parallel Computing Toolbox [17] (optional)

All of these toolboxes can be installed using the standard add-on tool "Add-On Explorer" [13] in Matlab. The Image Processing Toolbox is used for basic image processing functions, such as color space conversion and filtering, which is required for extracting patches from the video frames and computing some of the handcrafted features. The Statistics and Machine Learning Toolbox includes functions for training and testing a support vector regression (SVR) model. The Deep Learning Toolbox is required for training and testing the CNN spatial feature extractor. The pre-trained ResNet-50 model is used as a baseline model for the feature extractor. If supported by the hardware, a graphics processing unit (GPU) can be used to speed up the training process. In this case, the Parallel Computing Toolbox is also required.

2.2 (Step 1b) Dataset Preparation

Three different third-party datasets are used for training and testing; an image dataset for training the feature extractor, and two video datasets for training and testing the video quality model. For running the experiments described in this paper, all three datasets should first be downloaded, including the metadata files for the subjective quality scores.

First, the LIVE Challenge (CLIVE) image quality dataset [3] is used for training the CNN spatial feature extractor. CLIVE is available for download in [2]. At the time of writing, the dataset includes 1,162 image files under /Images folder, and three .mat files for metadata (*AllImages_release.mat*, *AllMOS_release.mat*, and *AllStdDev_release.mat*) under the /Data folder. All these files are required.

Two different video quality datasets have been used for training and testing the final CNN-TLVQM model: KoNViD-1k [7] and LIVE-VQC [20], available for download in [6] and [21], respectively. KoNViD-1k dataset comes in two parts, metadata and videos, one .zip file for each. The .zip files should be extracted and placed under the same folder. From this dataset, we use all the 1,200 video files in .mp4 format, but only one out of the three .csv files (*KoNViD_1k_attributes.csv*).

LIVE-VQC dataset contains 535 videos under the /Videos folder, all in .mp4 format, and one .mat file for metadata (*data.mat*), including subjective scores for each video file. All these files should be extracted and placed under the same folder.

3 EXPERIMENTS

CNN-TLVQM source code is available in [10], and for convenience, we have prepared scripts for easy replication of our earlier results. One can either run individual scripts to go step by step, or run all steps at once in a batch fashion.

We have prepared a master script that reproduces the results for CNN-TLVQM as well as CNN without the low complexity (LC) TLVQM features, as reported in Tables 1-3 in the original paper [23], assuming that the required datasets and Matlab modules have been properly installed. The master script requires four input parameters: 1) path to CLIVE dataset *clive_path*; 2) path to the KoNViD-1k dataset *konvid_path*; 3) path to the LIVE-VQC dataset *livevqc_path*; and 4) indicator for using either central processing unit (CPU) or GPU for training and testing the CNN model, set to either 'cpu' or 'gpu'. The script will perform simple tests to check if the datasets are installed correctly. Usage example of the script is the following:

```
masterScript(clive_path, konvid_path,
livevqc_path, 'gpu');
```

As an output, the script writes the results corresponding to Tables 1-3 in a comma-separated values (CSV) file *results.csv*. After the master script has been started, it will automatically go through steps 2-4 in Figure 1, as explained in the subsections below.

3.1 (Step 2) Training the CNN Feature Extractor

Before CNN feature extractor is trained, CLIVE dataset is preprocessed to generate patches of 224×224 pixels out of each image in the dataset. The resolution of most images is 500×500 pixels, and in this case, we extract nine partially overlapping patches. To further augment the training data, we also rotate each patch by 90, 180, and 270 degrees. As a result, we obtain 36 patches for each image. Each patch is assigned the same quality score as the original image. We use truncated Gaussian distribution to derive a probabilistic presentation of quality scores from the mean opinion score (MOS) values and their standard deviations, stored in file *LiveC_prob.mat*. The probabilistic presentation approximates the distribution of scores given by the users to the five different quality levels used for rating. When training the model, we use the probabilistic presentation as training target, instead of single value MOS. Given *in_path* is the path to the CLIVE dataset and *out_path* is the path to the resulting patches, the script for preprocessing can be used as follows:

```
processLiveChallenge(in_path, out_path);
```

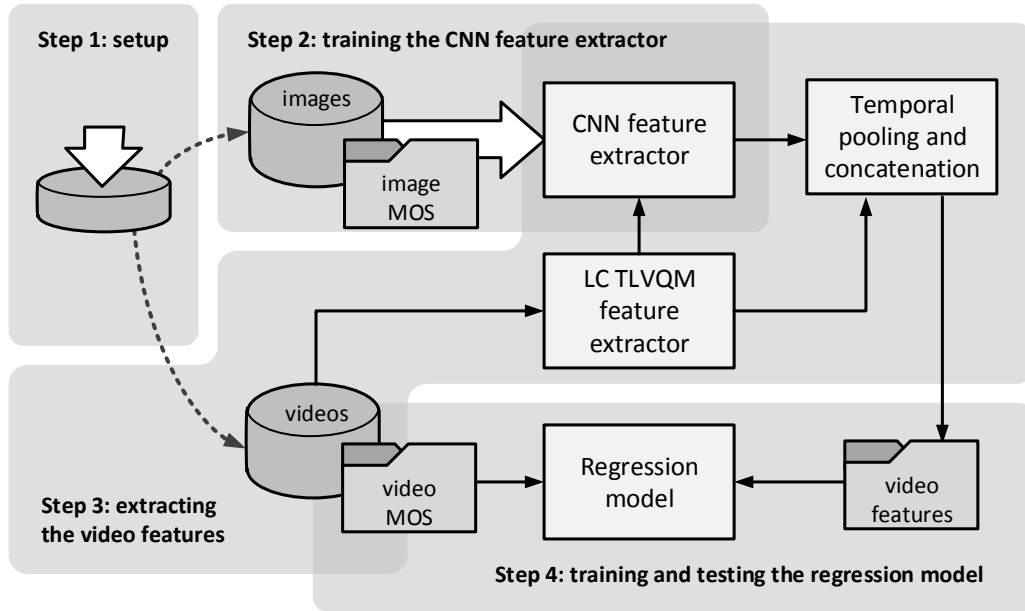


Figure 1: The main steps for the CNN-TLVQM reproducibility study.

ResNet-50 model pre-trained with ImageNet database for image classification is used as a baseline model. The fully connected (FC) and softmax layers for classification are removed and replaced by two new FC layers and Huber regression layer for predicting the distribution of subjective quality ratings. After re-training the modified model via transfer learning, the feature extractor is obtained by removing the last FC layer, and using the 128 outputs of the remaining FC layer as a feature vector. Given that path is the path to the folder with the patches (the same as out_path for script processLiveChallenge), output model file is cnnfile, and cpugpu is either 'cpu' or 'gpu', the model can be trained by calling the script as follows:

```
trainCNNmodel(path, cnnfile, cpugpu);
```

In masterScript, we use default file name CNN_model.mat' to store the trained model. This step can be skipped, if a pre-trained model is available; refer to the repository [10] for obtaining a pre-trained model. The training parameters are hardcoded in trainCNNmodel script, and it can be modified to test different training parameters.

3.2 (Step 3) Extracting Features from Videos

This step can be run directly if the pre-trained model exists in the same folder as the source code, or if the previous step has been run explicitly. Feature extraction is the most crucial and computationally complex part of the CNN-TLVQM architecture. For more details on the feature extraction process, the reader may refer to the original paper [11]. In the feature extraction phase, a data structure is generated, where one video sequence is represented by a sequence of feature vectors, and each vector represents a one-second segment

of the video. In this study, features for all the videos in KoNViD-1k and LIVE-VQC datasets are first extracted and saved in a file to be used later for training and testing the regression model to predict video quality. In masterScript we have defined default file names for KoNViD-1k and LIVE-VQC as KoNViD_features.mat and LIVEVQC_features.mat, respectively. We have implemented two scripts for extracting the features for the two datasets. Both scripts use four input parameters: path to the folder with the videos path, output feature file ftrt, input CNN feature extractor file cnnf (in masterScript, the file is CNN_model.mat), and cpugpu is either 'cpu' or 'gpu'. For KoNViD-1k dataset, use:

```
computeFeaturesForKoNViD1k(path, featurefile,
cnnfile, cpugpu);
```

For the LIVE-VQC dataset, use:

```
computeFeaturesForLIVEVQC(path, featurefile,
cnnfile, cpugpu);
```

To extract features from different video databases, the scripts for KoNViD-1k and LIVEVQC can be used as template, reflecting the particularities of the new dataset (e.g., format for the metadata, directory structure). The same pre-trained CNN model can be directly used on the new database.

3.3 (Step 4) Regression

When the features are available, different regression models can be used to predict MOS from the sequence of feature vectors. In this study, we test the model with SVR and long-short term memory (LSTM). In SVR, temporal average pooling is first employed to combine the feature vectors for each segment, and this feature vector is then used as input to SVR model. In LSTM, the sequence

of feature vectors is used directly as input to a simple recurrent neural network consisting of LSTM unit and FC layer as the head.

In this study, we use KoNViD-1k and LIVE-VQC databases to evaluate the performance of CNN-TLVQM in both single database and cross-database scenarios. In the single database scenario, the videos in the selected database are randomly split into training and testing sets (80% for training and 20% for testing). Regression models can be trained fast, because we use pre-computed features as input, and feature extraction is the most time-consuming part of the process. We repeat the training and testing procedure with 100 different splits and report the average MOS prediction accuracy and standard deviations in terms of Spearman rank order correlation coefficient (SRCC), Pearson linear correlation coefficient (PLCC), and root mean squared error (RMSE). Note that we use MOS values normalized to [0,1] during the training process, but have re-scaled RMSE to the original MOS scale in the reported results. Given that variable features contains data structure with the features for the selected video dataset, stored as defined in Step 3, and variable MOS contains the respective ground truth MOS, the usage examples for the scripts generating the results using SVR and LSTM regressors are as follows:

```
res = predictMOSwithSVR_100splits(features, mos);
res = predictMOSwithLSTM_100splits(features, mos);
```

In the cross-database scenario, we use first KoNViD-1k for training and LIVE-VQC for testing, and then vice versa. For this purpose, we have implemented a script that takes the training and testing features, as well as the corresponding MOS, as separate input parameters, as follows:

```
results = predictMOSwithSVR_CrossDB(features_train,
mos_train, features_test, mos_test);
```

The same scripts can also be used for testing the model on other datasets, assuming that the features are stored in the same format. For testing different regressors, these scripts can be used as templates to modify.

4 REVIEWING PROCESS

4.1 Reproducing the Results from the Original Paper

Using the pipeline described in this paper, the following results from the original paper can be reproduced:

- Table 1: Average PCC, SCC, RMSE and their standard deviations for the KoNViD-1k dataset.
- Table 2: Average PCC, SCC, RMSE and their standard deviations for the LIVE VQC dataset.
- Table 3: Cross-database results (trained on KoNViD-1k, tested on LIVE VQC, and vice versa).
- Figure 5: Predicted MOS vs. ground truth MOS using CNN-TLVQM in cross-database scenario (training with KoNViD-1k and testing with LIVE VQC). The script for generating a scatterplot from the final CSV is not included in the repository, but is relatively straightforward.

There have been a few minor updates to the repository [10] for ensuring cross-platform compatible directory addressing, consistent use of case, improved dataset sanity checking rules, typo-free

writing and correct input filenames. From the reviewers' perspective, as of commit 3813ca4, the experiments presented in [11] are possible to reproduce using the scripts provided in the official GitHub repository [10]. There are no missing requirements or additional packages needed to run the scripts, and adequate documentation and instructions can be found for all steps including pre-processing, training, feature extraction, and regression.

4.2 Compatibility Considerations

When we tested the implementation using Matlab version R2020b, we initially noticed that our results could not be reproduced accurately. The reason appeared to be different default weight initialization algorithms for the FC and LSTM layers in different Matlab versions. Therefore, we modified our implementation in the reproducibility package so that the weight initialization methods are defined explicitly. However, it is possible that other compatibility problems can occur in other Matlab versions, and therefore we recommend using either version R2018b or R2020b. Nevertheless, we would not expect drastic differences in the results. It should also be noted that due to random initializations when training the CNN model, the results will not be completely identical after different test runs; in any case, the results are supposed to be consistent with the conclusions in [11].

4.3 Runtime Benchmarks

We provide runtime benchmarks to serve as a baseline for future replication efforts. We use Matlab R2020b on Windows 10, with an NVIDIA GeForce RTX2080 Ti GPU. The runtime (in seconds) for different steps are listed in Table 1.

Step	Dataset	Runtime (s)
Step 2	CLIVE	1,536
Step 3	KoNViD-1k	30,248
	LIVE-VQC	37,249
Step 4	KoNViD-1k	20,579
	LIVE-VQC	15,674
	Cross-dataset	185

Table 1: Runtime (in seconds) for different steps.

5 CONCLUSION

In this paper, we document the replication process for "Blind Natural Video Quality Prediction via Statistical Temporal Features and Deep Spatial Features" published in ACM Multimedia 2020, where a two-level video quality model employing statistical temporal features and spatial features extracted by a deep CNN, called CNN-TLVQM, was proposed. The corresponding software repository and instructions have been revised in collaboration with the reviewers, and improved to facilitate the independent replication of all results.

ACKNOWLEDGMENTS

This work was supported in part by Natural Science Foundation of China under grant 61772348, and in part by Guangdong "Pearl River Talent Recruitment Program" under Grant 2019ZT08X603.

REFERENCES

- [1] P. Bianco, S. and Celona, P. Napoletano, and R Schettini. 2018. On the use of deep learning for blind image quality assessment. *Signal, Image and Video Processing* 12 (2018), 355–362. <https://doi.org/10.1007/s11760-017-1166-8>
- [2] D. Ghadiyaram and A. C. Bovik. 2015. *LIVE in the wild image quality challenge database*. Retrieved Jul 17, 2021 from <https://live.ece.utexas.edu/research/ChallengeDB/index.html>
- [3] D. Ghadiyaram and A. C. Bovik. 2016. Massive online crowdsourced study of subjective and objective picture quality. *IEEE Transactions on Image Processing* 25, 1 (2016), 372–387. <https://doi.org/10.1109/TIP.2015.2500021>
- [4] F. Götz-Hahn, V. Hosu, H. Lin, and D. Saupe. 2019. KonVid-150k: A Dataset for No-Reference Video Quality Assessment of Videos in-the-Wild. [arXiv:1912.07966](https://arxiv.org/abs/1912.07966) [cs.CV]
- [5] F. Götz-Hahn, V. Hosu, and D. Saupe. 2020. Critical analysis on the reproducibility of visual quality using deep features. [arXiv:2009.05369v2](https://arxiv.org/abs/2009.05369v2) [cs.CV]
- [6] V. Hosu, F. Hahn, M. Jenadeleh, H. Lin, H. Men, T. Szirányi, S. Li, and D. Saupe. 2017. *The Konstanz natural video database*. Retrieved Jul 17, 2021 from <http://database.mmsp-kn.de/konvid-1k-database.html>
- [7] V. Hosu, F. Hahn, M. Jenadeleh, H. Lin, H. Men, T. Szirányi, S. Li, and D. Saupe. 2017. The Konstanz natural video database (KoNViD-1k). In *Proceedings of IEEE International Conference on Quality of Multimedia Experience (QoMEX'17)*. Erfurt, Germany. <https://doi.org/10.1109/QoMEX.2017.7965673>
- [8] V. Hosu, H. Lin, T. Sziranyi, and D. Saupe. 2020. KonIQ-10k: An ecologically valid database for deep learning of blind image quality assessment. *IEEE Transactions on Image Processing* 29, 1 (2020), 4041–4056. <https://doi.org/10.1109/TIP.2020.2967829>
- [9] J. Korhonen. 2019. Two-level approach for no-reference consumer video quality assessment. *IEEE Transactions on Image Processing* 28, 12 (2019), 5923–5938. <https://doi.org/10.1109/TIP.2019.2923051>
- [10] J. Korhonen. 2020. *CNN-TLVQM reference implementation*. Retrieved Jul 17, 2021 from <https://github.com/jarikorhonen/cnn-tlvqm/>
- [11] J. Korhonen, Y. Su, and J. You. 2020. Blind Natural Video Quality Prediction via Statistical Temporal Features and Deep Spatial Features. In *Proceedings of ACM Multimedia (MM'20)*. Seattle, WA, USA. <https://doi.org/10.1145/3394171.3413845>
- [12] D. Li, T. Jiang, and M. Jiang. 2019. Quality assessment of in-the-wild videos. In *Proceedings of ACM Multimedia (MM'19)*. Nice, France. <https://doi.org/10.1145/3343031.3351028>
- [13] MathWorks. 2021. *Add-On Explorer*. Retrieved Jul 23, 2021 from <https://www.mathworks.com/products/matlab/add-on-explorer.html>
- [14] MathWorks. 2021. *Deep Learning Toolbox*. Retrieved Jul 23, 2021 from <https://www.mathworks.com/products/deep-learning.html>
- [15] MathWorks. 2021. *Deep Learning Toolbox Model for ResNet-50 Network*. Retrieved Jul 23, 2021 from <https://aaa.mathworks.com/matlabcentral/fileexchange/64626-deep-learning-toolbox-model-for-resnet-50-network>
- [16] MathWorks. 2021. *Image Processing Toolbox*. Retrieved Jul 23, 2021 from <https://www.mathworks.com/products/image.html>
- [17] MathWorks. 2021. *Parallel Computing Toolbox*. Retrieved Jul 23, 2021 from <https://www.mathworks.com/products/parallel-computing.html>
- [18] MathWorks. 2021. *Statistics and Machine Learning Toolbox*. Retrieved Jul 23, 2021 from <https://www.mathworks.com/products/statistics.html>
- [19] R. G. Nieto, H. D. B. Restrepo, R. F. Quintero, and A. C. Bovik. 2020. No reference video quality assessment with authentic distortions using 3-D deep convolutional neural network. In *Proceedings of IS&T International Symposium on Electronic Imaging (EI'20)*. Burlingame, CA, USA. <https://doi.org/10.2352/ISSN.2470-1173.2020.9.IQSP-168>
- [20] Z. Sinno and A. C. Bovik. 2018. Large scale subjective video quality study. In *Proceedings of IEEE International Conference on Image Processing (ICIP'18)*. Athens, Greece. <https://doi.org/10.1109/ICIP.2018.8451467>
- [21] Z. Sinno and A. C. Bovik. 2018. *LIVE video quality challenge database*. Retrieved Jul 17, 2021 from <https://live.ece.utexas.edu/research/LIVEVQC/index.html>
- [22] D. Varga, D. Saupe, and T. Sziranyi. 2018. A content preserving deep architecture for blind image quality assessment. In *Proceedings of IEEE International Conference on Multimedia and Expo (ICME'18)*. San Diego, CA, USA. <https://doi.org/10.1109/ICME.2018.8486528>
- [23] Z. Ying, M. Mandal, D. Ghadiyaram, and A. C. Bovik. 2020. ‘Patching up’ the video quality problem. [arXiv:2011.13544](https://arxiv.org/abs/2011.13544) [cs.CV]
- [24] J. You and J. Korhonen. 2019. Deep neural networks for video quality assessment. In *Proceedings of IEEE International Conference on Image Processing (ICIP'19)*. Taipei, Taiwan. <https://doi.org/10.1109/ICIP.2019.8803395>